题目：2020 至 2023 年青岛天气数据处理分析

一、任务描述

对青岛 2020 至 2023 年的天气情况进行处理分析。通过对天气，气温，风力风向等多方面的数据进行处理分析与数据可视化得到青岛天气的大致状况。主要对气温与天气状况进行分析，借此推断青岛是否为一个宜居的城市。

二、数据获取

1、数据描述

（1）数据来源：天气后报网站，如图 1 所示。

网址：http://www.tianqihoubao.com/lishi/qingdao/month/202301.html

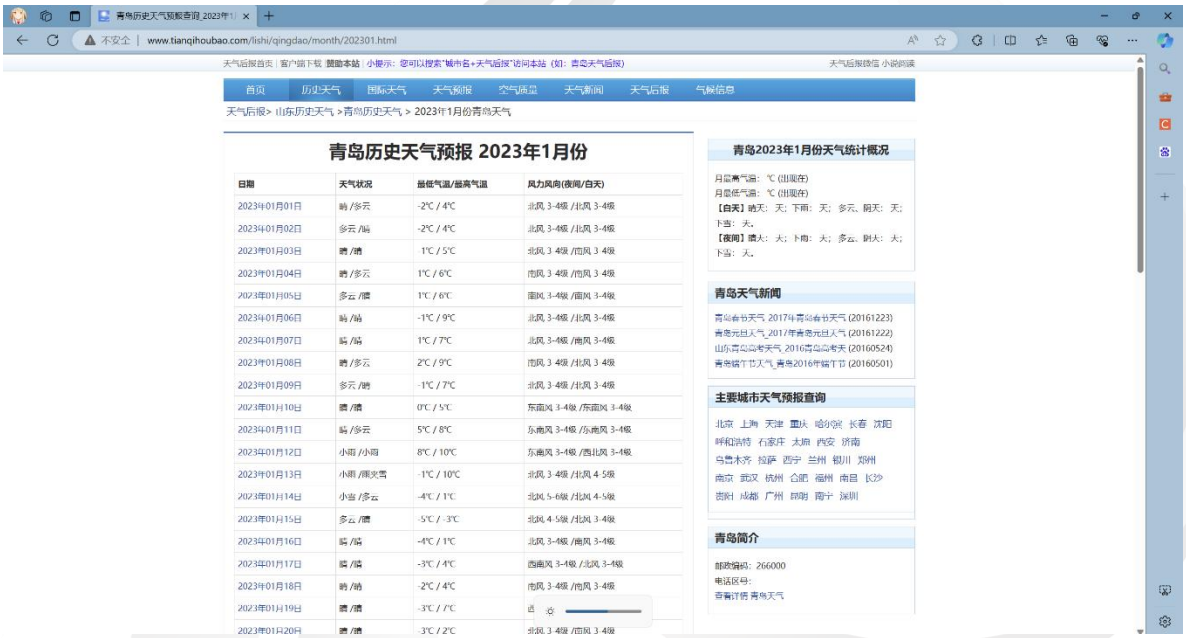

图 1　青岛历史天气预报 2023 年 1 月份

（2）数据获取：对网站中的天气状况、气温、风力风向数据进行获取，并写入 csv 文件中。

2、使用工具

主要使用 python 爬虫库 requests 与 html 代码处理库 BeautifulSoup 来获取并处理数据。

3、数据获取步骤

（1）对不同月份的网址进行分析可以发现，对于 yyyy 年 mm 月的网页，其网址为 http://www.tianqihoubao.com/lishi/qingdao/month/yyyymm.html。

于是可以通过遍历的方式对不同年月份的网址进行获取，如图 2。

```
for year in range(2023,2019,-1):
    for month in range(1,13):
        if month < 10:
            day = str(year) + '0' + str(month)
        else:
            day = str(year) + str(month)
        url = 'http://www.tianqihoubao.com/lishi/qingdao/month/'+ day + '.html'
```

图 2　遍历获取 2023 年至 2020 年的天气数据网址

（2）得到网址后，通过 requests 库中的 get 函数来获取网页源码，为了避免被网站反爬虫，利用了 fake_useragent 库来伪造浏览器的 User-Agent[1]。同时，在访问失败时会间隔随机时间后再次访问，模拟正式用户行为，如图 3。

```
def gethtml(url,head):  #获取网页源码
    session = req.Session()
    for i in range(5):  #获取失败后重试
        try:
            response = session.get(url, headers=head)
            response.raise_for_status()
        except req.exceptions.RequestException as e:
            print(f'Request failed: {e}')
            time.sleep(random.uniform(0.5,1.5))
        else:
            break
    return response.text
```

图 3　模拟正常用户访问网页获取源码（head 为伪造的 User-agent）

（3）得到源码后，利用 beautifulsoup 库来提取源码中的数据。通过解析源码可以发现数据被存储在一个 table 中。于是可以通过遍历 table 来获取数据，并写入 csv 文件中，如图 4。（其中存储文件夹以提前通过 os 库创建，betterdata 函数用于去除数据中多余的空格并修改日期格式。）

```
def wr_csv(html,day,year):   #将数据写入csv文件
    csv_name = 'Data/' + str(year) + '/' + day + '.csv'
    with open(csv_name, 'w', newline='',encoding='utf-8') as f:
        writer = csv.writer(f)
        soup = bs(html, 'html.parser')
        table = soup.find('table')
        tr = table.find_all('tr')
        data = []
        for x in tr:
            td = x.find_all('td')
            for y in td:
                data.append(betterdata(y.text.strip()))
            writer.writerow(data)
            data = []
```

图 4　获取数据并写入 csv 文件中

4、程序代码

（1） 最后通过 crawler 函数来进行所有操作。如图 5。

```python
def crawler():  #执行爬虫并存储数据
    dir()
    for year in range(2023,2019,-1):
        for month in range(1,13):
            head = {
    "User-Agent": ua.random,
    "Connection" : "cloes"
    }
            if month < 10:
                day = str(year) + '0' + str(month)
            else:
                day = str(year) + str(month)
            url = 'http://www.tianqihoubao.com/lishi/qingdao/month/'+ day + '.html'
            print('正在爬取' + str(year) + '年' + str(month) + '月' + '的数据')
            print('数据网址:' + url)
            html = gethtml(url,head)
            print(str(year) + '年' + str(month) + '月' + '的数据爬取完成')
            wr_csv(html,day,year)
    print('数据爬取完成')
```

图 5　执行爬虫（dir 函数用于创建文件夹）

（2） 程序运行时如图 6。

```
问题    输出    调试控制台    终端    端口

PS D:\化学2301贺锦翔> & C:/Users/yukino/AppData/Local/Programs/Python/Python39/python.exe d:/化学2301贺锦翔/GetData.py
正在爬取2023年1月的数据
数据网址:http://www.tianqihoubao.com/lishi/qingdao/month/202301.html
2023年1月的数据爬取完成
正在爬取2023年2月的数据
数据网址:http://www.tianqihoubao.com/lishi/qingdao/month/202302.html
2023年2月的数据爬取完成
正在爬取2023年3月的数据
数据网址:http://www.tianqihoubao.com/lishi/qingdao/month/202303.html
2023年3月的数据爬取完成
正在爬取2023年4月的数据
数据网址:http://www.tianqihoubao.com/lishi/qingdao/month/202304.html
```

图 6　执行程序

（3） 爬取到的数据如图 7。

Data > 2020 > 202001.csv > data

```
日期,天气状况,最低气温/最高气温,风力风向(夜间/白天)
2020-01-01,晴/多云,-1℃/4℃,西南风3-4级/西南风3-4级
2020-01-02,多云/多云,1℃/6℃,西北风3-4级/西北风3-4级
2020-01-03,晴/晴,1℃/6℃,北风3-4级/北风3-4级
2020-01-04,晴/多云,2℃/7℃,北风3-4级/北风3-4级
2020-01-05,小雨/小雨,3℃/7℃,东风3-4级/东风3-4级
2020-01-06,小雨/小雨,4℃/8℃,东北风4-5级/东北风4-5级
2020-01-07,雨夹雪/小雪,0℃/4℃,北风6-7级/北风6-7级
2020-01-08,晴/晴,-4℃/3℃,北风5-6级/北风5-6级
2020-01-09,晴/晴,-1℃/3℃,北风3-4级/北风3-4级
2020-01-10,晴/多云,-1℃/5℃,西北风3-4级/西北风3-4级
```

图 7　部分数据

三、数据预处理

1、预处理目的

通过爬虫所获取的数据均为字符串形式，且数据分类不全面。对数据分析带来了许多阻碍，于是先对数据进行预处理，便于后续的分析与可视化。

2、所用工具

主要使用 pandas 库进行预处理。

3、预处理步骤

（1）首先读取爬取到的数据，并按照年份分类存储于字典中，同一年份的数据存储于列表中。如图 8。

```
def ReadData(): #读取数据
    total_data = {}
    for year in range(2020,2024):
        year_data = []
        for month in range(1,13):
            if month < 10:
                filename = 'Data/' + str(year) + '/' + str(year) + '0' + str(month) + '.csv'
            else:
                filename = 'Data/' + str(year) + '/' + str(year) + str(month) + '.csv'
            if os.path.exists(filename):
                data = pd.read_csv(filename,encoding='utf-8')
                year_data.append(data)
        total_data[year] = year_data #这里数据按月存储(列表索引=月份-1)
    return total_data
```

图 8　读取数据

（2）为了便于对整年的数据进行分析，对按月存储的数据进行聚合，并存入 excel 文件中。如图 9。

```
def aggregatedata(data): #聚合数据（来自ReadData函数）
    new_data = deepcopy(data)  #不将原数据改变
    for year in range(2020,2024):
        new_data[year] = pd.concat(new_data[year],ignore_index=True)
    return new_data #这里数据按年存储

def MakeExcel(aggregate_data):  #将数据写入Excel
    try:
        with pd.ExcelWriter('2020年到2023年青岛天气数据.xlsx') as writer:
            for year in range(2020,2024):
                aggregate_data[year].to_excel(writer,index=False,sheet_name=str(year))
    except:
        pass
```

图 9　聚合数据并存入 excel

（3）为了后续分析数据与将数据存入数据库，创建了数据类。数据类的主要功能有：空

值检查、预处理数据。同时设置了查询数据方法，后续操作只需从 PertreatData.py 文件中引用实例化的数据类即可读取任意年月的数据。下面分点对类中功能进行介绍。

①预处理数据。为了方便后续操作，类中保留了原始数据，同时也对原始数据进行了处理，主要操作为：将天气与风力风向分割为白天与夜间、将最高气温与最低气温分割开并转化为数字类型、添加平均气温（最高气温与最低气温的平均值）。具体代码实现可见 PertreatData.py 文件中 betterdata 函数，处理后的数据结构如图 10。

	白天天气	夜间天气	最低气温	最高气温	平均气温	夜间风力风向	白天风力风向
日期							
2020-01-01	晴	多云	-1	4	1.5	西南风3-4级	西南风3-4级
2020-01-02	多云	多云	1	6	3.5	西北风3-4级	西北风3-4级
2020-01-03	晴	晴	1	6	3.5	北风3-4级	北风3-4级
2020-01-04	晴	多云	2	7	4.5	北风3-4级	北风3-4级
2020-01-05	小雨	小雨	3	7	5.0	东风3-4级	东风3-4级
...	...	...	...	...	...	...	...
2020-12-27	多云	多云	2	9	5.5	北风3-4级	北风3-4级
2020-12-28	多云	中雪	-1	9	4.0	东北风5-6级	东北风5-6级
2020-12-29	中雪	阴	-9	-2	-5.5	北风5-6级	北风5-6级
2020-12-30	多云	晴	-9	-6	-7.5	西北风4-5级	西北风4-5级
2020-12-31	晴	晴	-8	-2	-5.0	西北风3-4级	西北风3-4级

369 rows × 7 columns

图 10　预处理后数据

②空值检查：为了避免空值的出现，在分析数据之前进行了空值检查。具体操作为：对于数字类型的列中空值，用整列的平均值进行填充。其他类型，用上一行或下一行的数据进行填充（考虑到连续几天的天气总是趋向于相同，这也不失为一种合理的方法）[2]。具体代码实现见 PertreatData.py 文件中 check_na 函数。

③数据获取：通过类方法来进行数据的获取，直接调取类方法即可获取所需的数据，如图 11。

```
def get(self,year,month=0):    #获取月或年数据(分析时使用)
    if month == 0:
        return self.year_data[year]
    else:
        return self.month_data[year][month-1]

def get_original(self,year,month=0):    #获取原始月或年数据(存入数据库)
    if month == 0:
        return self.original_year_data[year]
    else:
        return self.original_month_data[year][month-1]
```

图 11　获取数据函数

四、数据分析与可视化展示

1、数据分析与可视化概要

在这一部分，主要运用 pandas 库和 matplotlib 库对前面所得到的数据进行分析与可视化。围绕主要的天气信息，通过将其转化为便于量化的指标，对青岛天气状况做出大致的分析，从而得出结论。

2、使用工具

主要使用 pandas 库对数据进行分析，matplotlib 库进行可视化。

3、数据分析步骤

（1）数据指标量化。对每日平均气温，每日温差，每日风力数据进行量化。分为多个等级。其中平均气温分为：冷、凉、舒适、热、非常热，共五个指标；温差分为：小、中、大，共三个指标；风力数据分为：轻风、和风、强风、大风、狂风[3]，共五个指标。具体代码实现见 AnalyseData.py 中 analyse 函数。通过多重条件判断进行指标判断。

（2）数据分析。通过预处理阶段实例化的 Data 类获取数据后，对数据进行遍历分析，同时利用字典保存分析结果，供后续可视化使用。分别对年数据与月数据进行了分析，下面分项介绍。

①年数据：对于每年的数据，主要从以下几个方面进行了分析：天气状况、气温与温差量化状况、风向与风力量化状况、整体天气状况、零下天数与最高，最低气温。将不同年份的分析结果存储于字典中，每一项分析数据存储于当年的字典中，以项目类型为键，供后续可视化使用。具体代码实现见 AnalyseData.py 中 analyse 函数。具体指标见图 12。

```
year_ana[year] = {
    '白天天气':{},
    '夜间天气':{},
    '温度':{},
    '温差':{},
    '白天风向':{},
    '白天风力':{},
    '夜间风向':{},
    '夜间风力':{},
    '未变天气':0,
    '未变风向':0,
    '零下天数':0,
    '温度总体':'',   #后续会用describe()方法的返回值填充
    '最高温':'',     #元组(最高温,日期)
    '最低温':''      #元组(最低温,日期)
}
```

图 12　年分析指标

②月数据：对于每月的数据，分析项目较年数据少些，但侧重点有所不同。主要对天气状况、温度总体、零下气温与最高，最低气温进行分析。同时在可视化时会绘制当月每日气温折线图。具体指标见图 13。

```
month_data = data.get(year,month)
month_ana[(year,month)] = {
'白天天气':{},
'夜间天气':{},
'零下天数':0,
'温度总体':'',   #后续会用describe()方法的返回值填充
'最高温':'',     #元组(最高温,日期)
'最低温':''      #元组(最低温,日期)
}
```

图 13　月分析指标

（3）数据可视化：利用 matplotlib 库对分析得到数据进行可视化。对于年数据，绘制了风力情况、风向情况、天气情况、温差情况、温度情况。对于月数据，绘制了天气状况与每日温度。具体代码实现见 AnalyseData.py 中 dr_pic 函数。下面展示部分图像，全部图像可于分析结果文件夹中找到。

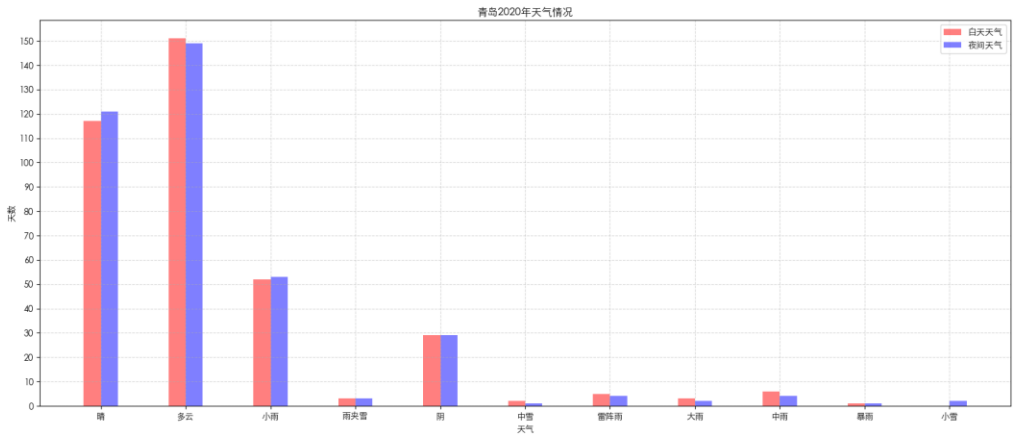

图 14　2020 年青岛天气情况

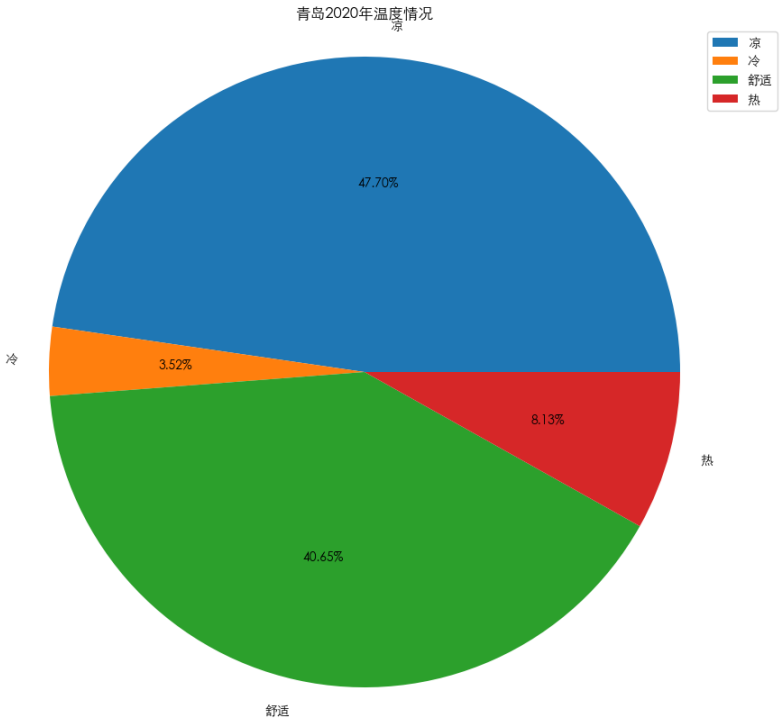

图 15　2020 年青岛温度情况

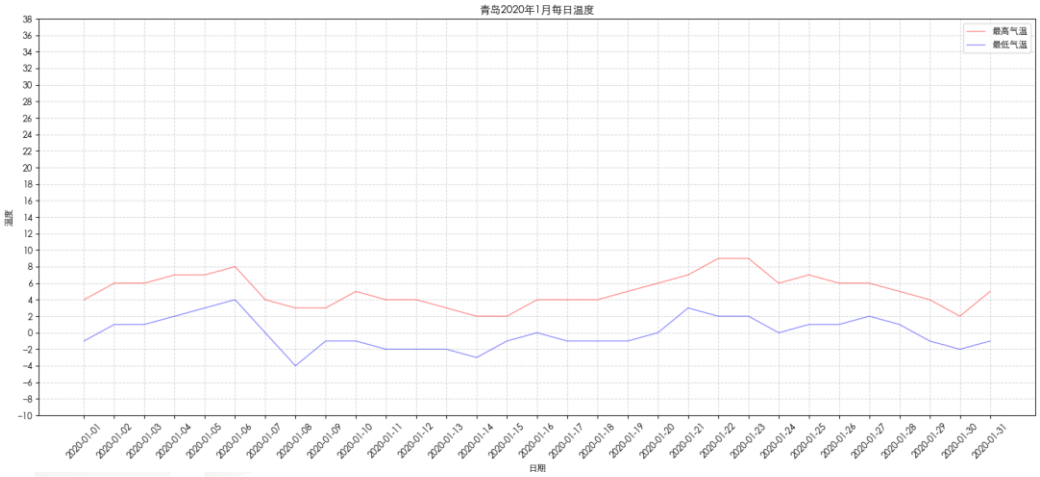

图 16　2020 年 1 月青岛每日温度

（4）总体情况结果写入 txt：对于总体分析结果，由于其不便于进行可视化，于是将其写入到了 txt 文件中。具体代码实现见 AnalyseData.py 文件 wr_txt 函数，结果见图 17 与图 18。

```
-------------------总体温度------------------
平均温度：14.10
%25分位数：6.0
%50分位数：15.0
%75分位数：22.5
最高温度：31(2020-08-17)
最低温度：-9(2020-12-29)

-------------------天气情况------------------
整日天气未变天数：183
零下天数：38
```

图 17　2020 年青岛天气总体情况

```
-------------------总体温度------------------
平均温度：2.52
%25分位数：1.25
%50分位数：2.0
%75分位数：3.75
最高温度：9(2020-01-22)
最低温度：-4(2020-01-08)

-------------------天气情况------------------
零下天数：15
```

图 18　2020 年 1 月青岛天气总体情况

4、分析结果

通过以上的分析可得知，青岛全年天气大多为晴天或多云，且天气状况稳定。气温大多处于舒适或偏凉的区间，且全年最高气温不超过 35 摄氏度。但青岛的冬季较为寒冷，且全年零下天数大多超过 30 天，不过考虑到青岛作为北方城市，冬季室内提供有供暖，室内温度也可以维持在舒适的区间。

综上，通过对青岛近几年的天气状况的分析，可以得出结论，青岛是一个宜居的城市。

五、数据管理展示

1、数据管理概述

考虑到可能的后续添加数据的需求，利用数据库进行数据存储可以大大减轻后续添加数据的工作量。于是使用 MySQL 数据库对数据进行存储，同时运用 pymysql 库来操作数据库。

2、使用工具

使用 pymysql 库对数据库进行命令操作，DataGrip 软件实现数据库可视化。

3、主要功能展示

通过 pymysql 库实现了数据库的增删改查功能，下面分条展示。

①链接数据库并写入原始数据。在创建“青岛天气数据”数据库后，利用 pymysql 库的 connect 函数链接到数据库并写入原始数据。具体代码实现见 SaveData.py 文件中 sql 类。代码运行结果如图 19、图 20。

```
from SaveData import sql
sql = sql(host='localhost',user='root',password='666666',port=3306,db='青岛天气数据',charset='utf8mb4')
sql.add_original_data()
```

✓ 3.8s

```
成功连接到数据库
正在添加已有数据
创建2020年青岛天气数据表成功
正在向2020年青岛天气数据表中插入数据
向2020年青岛天气数据表中插入数据成功
创建2021年青岛天气数据表成功
正在向2021年青岛天气数据表中插入数据
向2021年青岛天气数据表中插入数据成功
创建2022年青岛天气数据表成功
正在向2022年青岛天气数据表中插入数据
向2022年青岛天气数据表中插入数据成功
创建2023年青岛天气数据表成功
正在向2023年青岛天气数据表中插入数据
向2023年青岛天气数据表中插入数据成功
已有数据已添加完成
```

图 19　代码运行结果

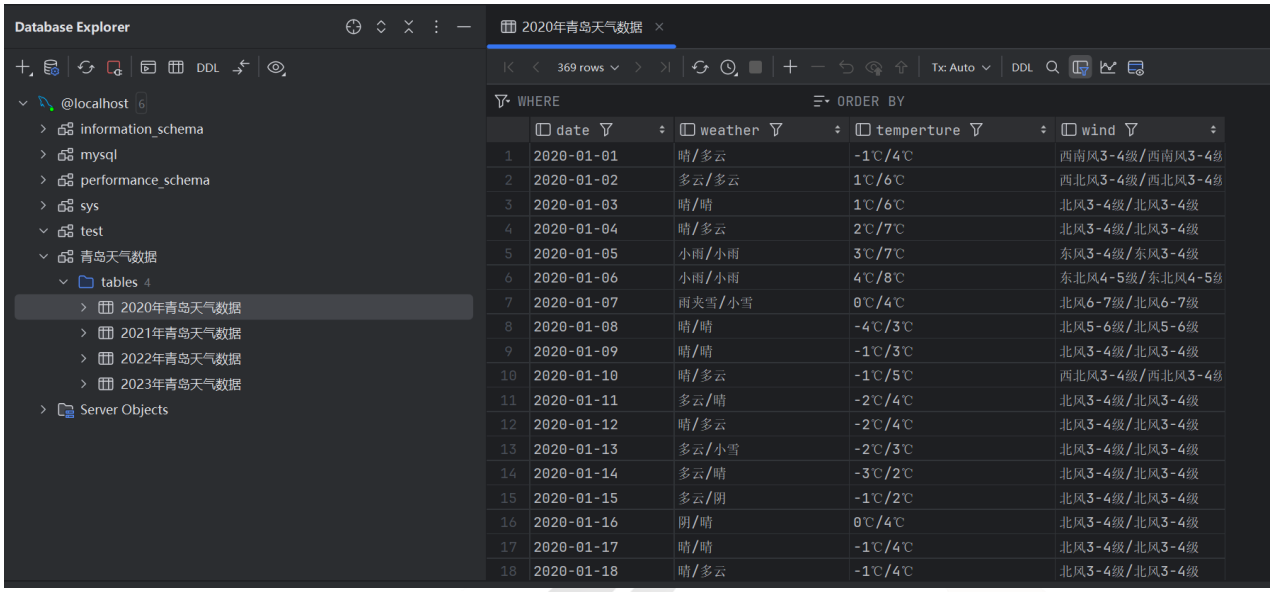

图 20　写入表后的数据库

②删除表中的数据。通过指定年份与日期删除对应日期的数据。具体代码实现见 SaveData.py 文件。运行结果如图 21、图 22。

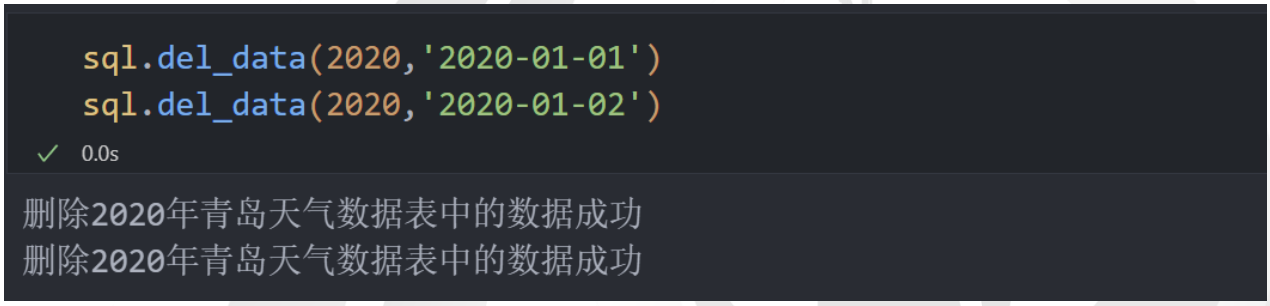

图 21　代码运行结果

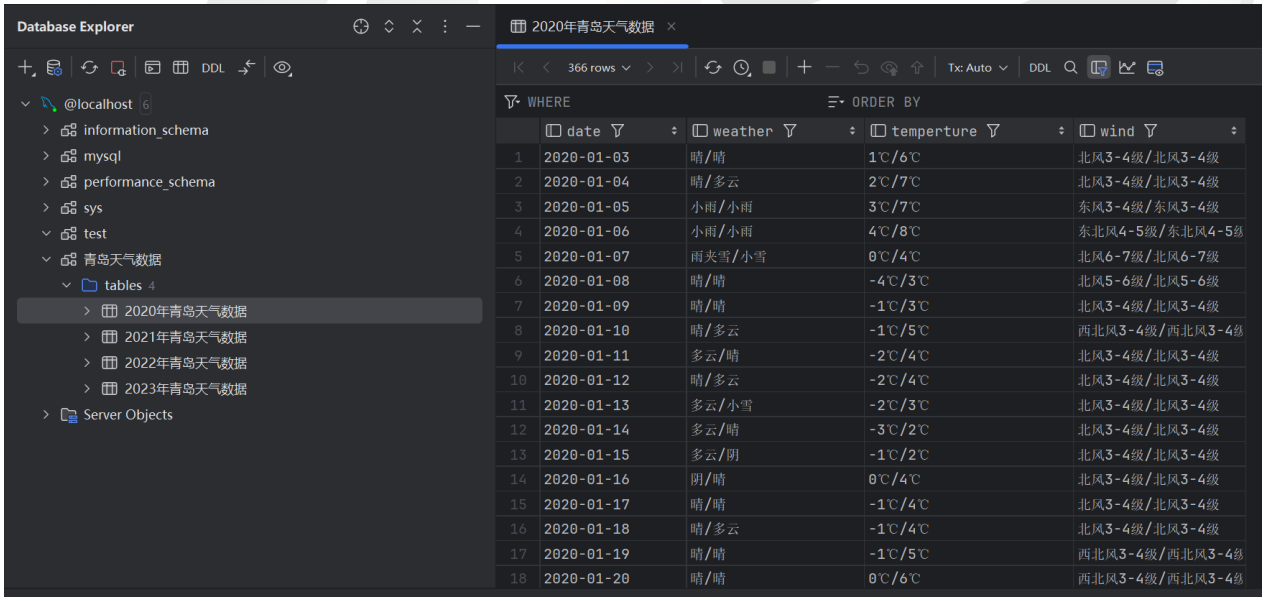

图 22　删除数据后的表

③删除整个表。通过指定年份删除存有对应年份数据的表。代码运行结果如图 23、图 24。

```
sql.del_table(2020)
[4]  ✓ 0.0s
…  删除2020年青岛天气数据表成功
```

图 23　代码运行结果

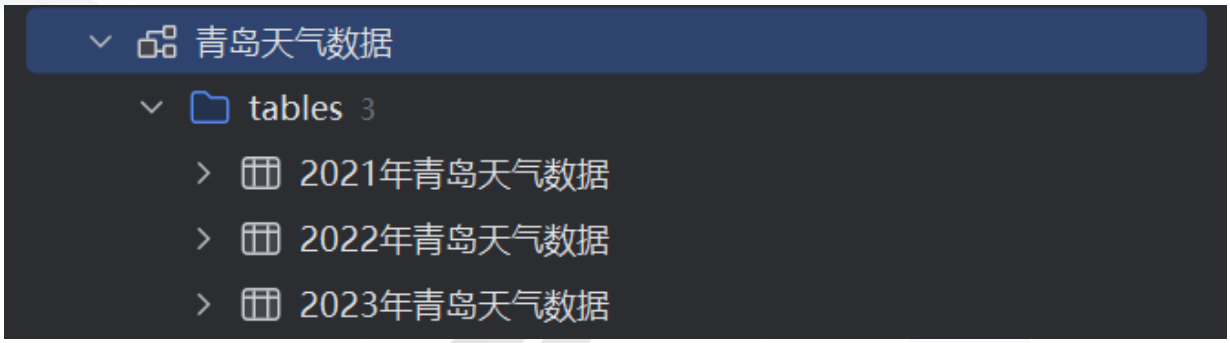

图 24　删除 2020 年数据表后的数据库

④修改表中数据。通过传入数据并指定表和对应日期的方式对表中对应日期的数据进行修改。代码运行结果见图 25、图 26。

```
data = ['1','2','3']
whe = '2021-01-01'
sql.change_data(2021,data,whe)
✓ 0.0s
修改2021年青岛天气数据表中的数据成功
```

图 25　代码运行结果

2021年青岛天气数据

367 rows　Tx: Auto　DDL

WHERE　ORDER BY

	date	weather	temperture	wind
1	2021-01-01	1	2	3
2	2021-01-02	晴/多云	-2℃/3℃	北风3-4级/北风3-4级
3	2021-01-03	多云/多云	0℃/5℃	北风3-4级/北风3-4级
4	2021-01-04	多云/多云	-2℃/5℃	北风3-4级/北风3-4级
5	2021-01-05	晴/晴	-8℃/0℃	北风3-4级/北风3-4级
6	2021-01-06	多云/多云	-14℃/0℃	西北风5-6级/西北风5-6级
7	2021-01-07	多云/晴	-15℃/-10℃	西北风5-6级/西北风5-6级
8	2021-01-08	晴/晴	-9℃/-4℃	西北风4-5级/西北风4-5级
9	2021-01-09	晴/晴	-7℃/-1℃	西北风3-4级/西北风3-4级
10	2021-01-10	晴/阴	-5℃/1℃	西北风3-4级/西北风3-4级
11	2021-01-11	多云/晴	-4℃/2℃	西北风3-4级/西北风3-4级
12	2021-01-12	晴/晴	-1℃/4℃	西南风3-4级/西南风3-4级
13	2021-01-13	晴/晴	3℃/9℃	北风3-4级/北风3-4级
14	2021-01-14	晴/多云	3℃/9℃	东南风3-4级/东南风3-4级
15	2021-01-15	多云/晴	-5℃/5℃	北风3-4级/北风3-4级
16	2021-01-16	晴/晴	-5℃/1℃	西北风3-4级/西北风3-4级
17	2021-01-17	晴/晴	-2℃/4℃	西南风3-4级/西南风3-4级
18	2021-01-18	晴/晴	0℃/6℃	北风3-4级/北风3-4级

图 26　修改后的表

⑤创建新表并写入数据。通过指定年份和传入 DataFrame 类型数据的方式创建数据表并写入数据。代码运行结果见图 27、图 28。

```
data = sql.data.get_original(2023)
sql.add_data(2024,data)
```

✓ 0.4s

```
创建2024年青岛天气数据表成功
正在向2024年青岛天气数据表中插入数据
向2024年青岛天气数据表中插入数据成功
```

图 27　代码运行结果

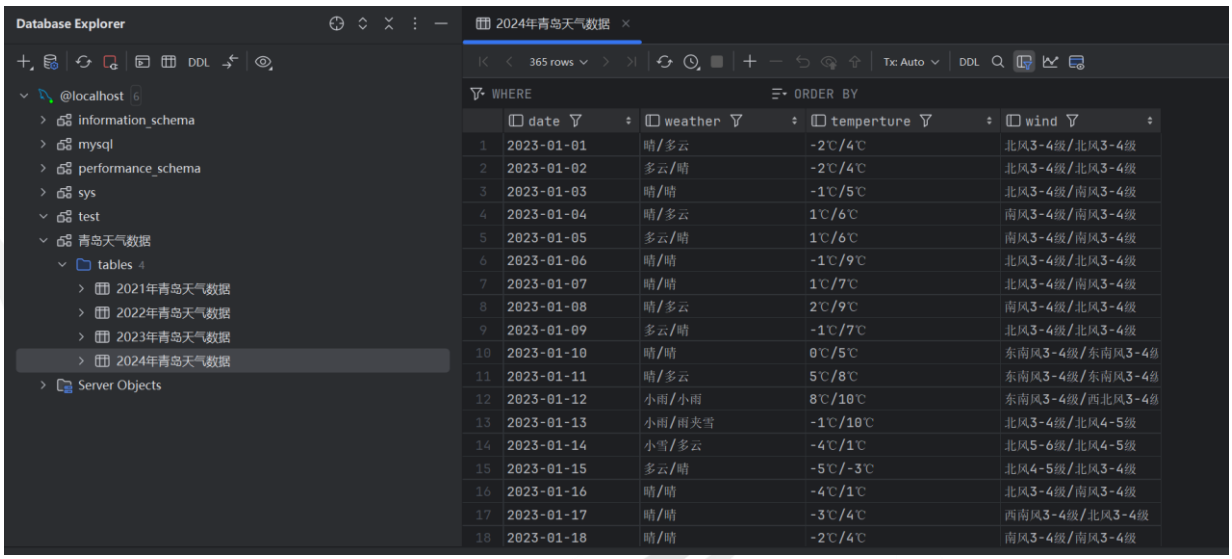

图 28　添加新表后的数据库

⑥查询数据。共提供了五种查询数据的方法。分别为：指定年份返回全年数据、指定年份与月份返回整月数据、指定年份与日期查询对应日期数据、指定年份与天气查询所有具有所指定天气的数据、查询所有气温小于零度或大于三十度的数据（考虑到查询指定气温范围数据的需求不常见，故不再通过存储数字类型的气温来提供具体气温范围数据的查询，而是利用字符串的模糊查询来提供极端气温数据的查询）。下面展示通过指定天气的方式进行查询，见图 29。其他查询方法可通过 find_data 方法实现，具体传入数据可见注释。

```
sql.find_data(year = 2021,weather='晴')
```

✓ 0.0s

日期	天气状况	最低气温/最高气温	风力风向
2021-01-02	晴/多云	-2℃/3℃	北风3-4级/北风3-4级
2021-01-05	晴/晴	-8℃/0℃	北风3-4级/北风3-4级
2021-01-07	多云/晴	-15℃/-10℃	西北风5-6级/西北风5-6级
2021-01-08	晴/晴	-9℃/-4℃	西北风4-5级/西北风4-5级
2021-01-09	晴/晴	-7℃/-1℃	西北风3-4级/西北风3-4级
...	...	...	...
2021-12-27	晴/晴	-4℃/5℃	西北风3-4级/西北风3-4级
2021-12-28	晴/晴	-1℃/5℃	西南风3-4级/西南风3-4级
2021-12-29	晴/晴	-3℃/5℃	西北风3-4级/西北风3-4级
2021-12-30	晴/晴	-3℃/5℃	西北风3-4级/西北风3-4级
2021-12-31	晴/多云	0℃/4℃	南风3-4级/南风3-4级

172 rows × 3 columns

图 29　指定天气数据查询

六、遇到的问题与解决方法

1、遇到的问题

（1）在进行数据分析时发现字符串类型数据分析较为困难。

（2）进行数据分析时难以以较为直观的方式表现出天气状况。

2、解决办法

（1）通过对数据进行预处理来简化数据分析时的工作量

（2）通过对气温与风力数据进行量化，得到了较为直观的天气信息。

参考文献

[1] CSDN.fake_useragent 生成随机请求头 UserAgent.https://blog.csdn.net/nikeylee/article/details/109570245

[2] 和鲸社区.Python 之以温度为例的气象数据缺测处理方法.https://www.heywhale.com/mw/project/5f2920d3d278b1002c261bc6

[3] 百度百科.风级.https://baike.baidu.com/item/%E9%A3%8E%E7%BA%A7/1699387#2-2